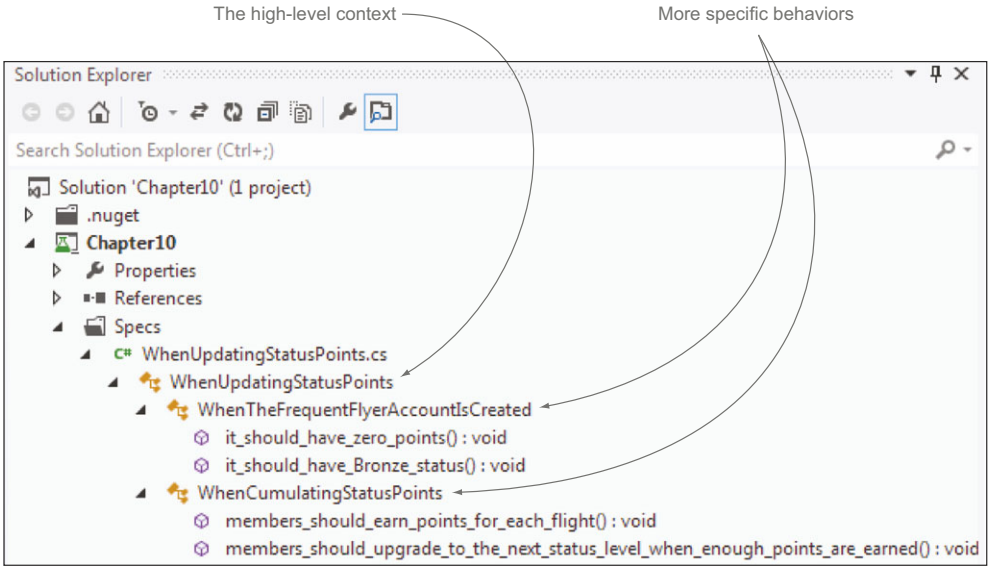


Figure 11.12 Structuring your unit tests well can make them read like living documentation within your IDE.

This approach dates from the early days of unit testing. One of the original motivations was that Java classes in a given package could access protected fields of other classes in that package, so if the unit test class was in the same package as the class being tested, you could access protected fields as part of the unit tests.

From a BDD perspective, the argument for this approach is less compelling. A BDD-style unit test should provide a worked example of how to use a class and how the class is expected to behave. If you need to access protected variables, you may be binding your test code too tightly to the implementation, which runs the risk of making the unit tests more brittle.

This approach also assumes a very tight coupling between the classes under test and the features (or behavior) those classes are implementing. If you refactor a class, change its name, or break it apart into several smaller classes, for example, the requirement that these classes implement shouldn't change.

Having a test class for each production class can also hamper refactoring. A modern IDE will tell you at a glance if a method or a class isn't being used anywhere. Unless it's part of an API for an external client, a method that's never used anywhere can generally be deleted, which results in less code to maintain going forward.

For all these reasons, many organizations apply a looser association between test classes and production classes. For example, some teams find that test packages or directories organized in terms of functional slices are often easier to navigate, especially when you come back to a code base after a long period.